

Static Reconstruction of `iboot_dec.bin`

Image Layout, Startup, Command Dispatch, and Boot Policy

Analysis of the supplied binary

September 13, 2026

Abstract

The supplied 3,699,832-byte artifact identifies itself as `mBoot-18000.162.8`, with an iBoot banner for `v59` and a `RELEASE` build marker. Its initial instructions decode as little-endian AArch64. This report reconstructs the preferred address mapping, relocation and initialization sequence, a shared memory-range predicate, command dispatch, and portions of automatic boot and recovery behavior. It also catalogs 16 command records and 62 other named callback records. Every aligned word is preserved in an accompanying annotated linear-sweep listing, while machine-readable indexes support further analysis. A follow-up logic audit records two conditional arithmetic-risk candidates and explicit remaining coverage gaps.

Scope: this is a partial semantic reconstruction, not the original source code or a complete reimplementaion. Recovered machine instructions, structural inferences, and unresolved behavior are explicitly distinguished. The binary was not executed, patched, or authenticated.

Contents

1	Artifact identity and evidence standard	3
1.1	Observed identity	3
1.2	What the confidence labels mean	3
2	Method and coverage	3
3	Address mapping and image layout	4
3.1	Preferred base and an important length discrepancy	4
3.2	Observed regions	5
4	Startup reconstruction	5
4.1	Entry and pre-relocation hardware operation	5
4.2	Overlap-aware relocation	7
4.3	Zero initialization and runtime entry	7

5	Command records and dispatch	8
5.1	Recovered descriptor family	8
5.2	Dispatch corroborates the handler field	9
5.3	Behavior supported by inspected handlers	10
6	A locally reconstructed memory-range predicate	10
7	Automatic boot, recovery, and policy tables	11
7.1	Automatic boot has concrete input and dispatch gates	11
7.2	Boot-policy actions form a distinct descriptor family	12
7.3	Recovery starts named worker-like tasks	12
8	Other reconstructed subsystem entry points	12
8.1	Image-related callbacks	12
8.2	Image4 and boot-policy evidence	13
8.3	Storage, hardware, and runtime evidence	14
9	Deliverables and reproducibility	14
10	Logic-bug audit	16
10.1	Result and limits of the claim	16
10.2	Audit coverage	16
10.3	C01: autoboot delay narrows before comparison	17
10.4	C02: register-value conversion truncates before a width check	18
10.5	Suspicious patterns not promoted to findings	19
10.6	Unresolved liveness and semantic risks	20
10.7	Reproducible checks and artifacts	20
11	What remains before a complete reconstruction	21
A	Hardware-tuning callback inventory	23

1 Artifact identity and evidence standard

1.1 Observed identity

Property	Observation
Input	<code>iboot_dec.bin</code>
Length	3,699,832 bytes; 0x387478
Banner at 0x200	iBoot for v59, copyright 2007–2026, Apple Inc.
Build style at 0x240	RELEASE
Build tag at 0x280	mBoot-18000.162.8
Instruction encoding	Little-endian AArch64; authenticated instructions present
Candidate preferred base	0x1fc080000

The SHA-256 of the complete supplied file is:

```
48ff474efdb852d15995fdcb00bbc17e040267fa94cde6d4304c8d8fca8931ef
```

The banner is *self-identification*, not signature verification or proof of provenance. The bytes establish the stated build tag, not the marketed device, operating-system release, or exact compilation date. A source-path string at 0x2a26c4 contains `drivers/apple/silicon/tunableh/t8140/tunableh.c`; this is a hardware-family clue, not proof of a particular retail model.

The file begins with executable-looking instructions rather than an ELF or Mach-O header. It is analyzed as a raw image with embedded data. The original container, extraction procedure, symbols, link map, and external firmware are not supplied. The filename suffix `_dec` alone does not establish how it was decrypted or whether all embedded objects are plaintext.

1.2 What the confidence labels mean

Observed: A byte string, literal, decoded instruction, or table field exists at a stated file offset.

Reconstructed: A local computation or control-flow relationship follows from inspected instructions; names and pseudocode are analytical labels.

Inferred: A subsystem role is suggested by named records, string references, or surrounding code, but is not exhaustively traced.

Unresolved: Original types, runtime states, indirect targets, signatures, or hardware behavior are not established by this analysis.

Unless prefixed by “VA,” every hexadecimal location below is a *file offset*. Evidence comes from the supplied binary and the generated indexes. External references provide architectural background only.

2 Method and coverage

The extractor `analysis/reconstruct.py` reads the input, verifies its hash, and uses the installed LLVM 19 AArch64 decoder through the LLVM C interface [1]. The decoder is configured for

`aarch64-apple-darwin` with `+v8.3a`. This enables common authenticated instructions but does not prove the exact target CPU or explain every Apple-specific system register. No dependencies were installed and no binary code was executed.

Mechanical inventory	Count
Aligned 4-byte words in the entire file	924,958
Printable runs of at least four characters	11,380
PACIASP/PACIBSP words in the code-dense prefix	8,860
Candidate entries from prologues, direct calls, and commands	16,639
Immediate BL sites with in-prefix targets	96,216
Plain BR/BLR sites in that prefix	256
Recovered string-reference records	5,152
Command records of the identified descriptor family	16
Named callback records across four families	78

These counts are *not* counts of verified source functions, reachable calls, or implemented user features. Linear scanning can interpret data as instructions. PAC markers miss leaf routines and may occur after a true entry; BL targets may identify shared fragments. The direct-call index’s “nearest preceding candidate” is only an indexing aid, not a recovered function boundary. Authenticated indirect branches such as `BLRAAZ` are not included in the plain BR/BLR count.

String references are intentionally conservative: the extractor recognizes adjacent `ADRP+ADD` address constructions and aligned absolute pointers. It does not perform general register propagation, follow every pointer, or resolve all references through metadata. Printable runs may include accidental text and are not a symbol table.

3 Address mapping and image layout

3.1 Preferred base and an important length discrepancy

The literal at `0x300` is `0x1fc080000`. At `0x24`, the entry code loads that literal into `x1`; `x0` is formed from the current code page at `0x1c–0x20`. After an early hardware helper, the code compares these addresses and takes a relocation path if they differ. This directly supports

$$\text{candidateVA}(\text{offset}) = 0x1fc080000 + \text{offset}.$$

Independent pointers to the command strings and handlers resolve consistently under the same mapping.

The end literal at `0x308` is `0x1fc407480`. Therefore the linked span is `0x387480`, whereas the supplied file ends at `0x387478`:

$$0x387480 - 0x387478 = 8 \text{ bytes}.$$

The relocation code rounds this linked span to a 64-byte multiple; it is already aligned. The discrepancy could reflect extraction or layout padding, but its cause is not established. No missing bytes have been invented. A loader or emulator must account for this rather than assuming file length equals the entire in-memory image span.

3.2 Observed regions

All ranges in this table are half-open; broad data boundaries are descriptive, not recovered linker sections.

File-offset range	Interpretation and evidence
0x0–0x1d4	Entry, relocation, copying, zeroing, and initial register setup.
0x200–0x300	Banner and build fields, with padding.
0x300–0x388	Literal addresses used by startup.
0x388–0x201b28	Predominantly code, but includes literal pools, jump tables, and embedded text.
0x201b28–0x29b2c0	Pointer-rich metadata and tables. At 0x201b28, a data record follows the last inspected instruction at 0x201b24.
0x29b2c0–0x2e0000	Numerous diagnostic strings and mixed constants; certificate-related names occur near 0x2de000.
Approximately 0x2e0000–0x330000	High-entropy data, with mixed edges. Its format is unresolved.
0x330000–0x387478	More metadata, named callback objects, device properties, strings, and zero-rich regions.

The windows beginning at 0x2e0000, 0x2f0000, 0x300000, and 0x310000 have entropy close to 7.99 bits per byte. Compression, encryption, and other densely encoded data can all produce this observation; the plot does not distinguish them. Conversely, successful instruction decoding does not establish executable code: LLVM decodes 230,802 words *after* the code-dense prefix, much of which visibly contains data.

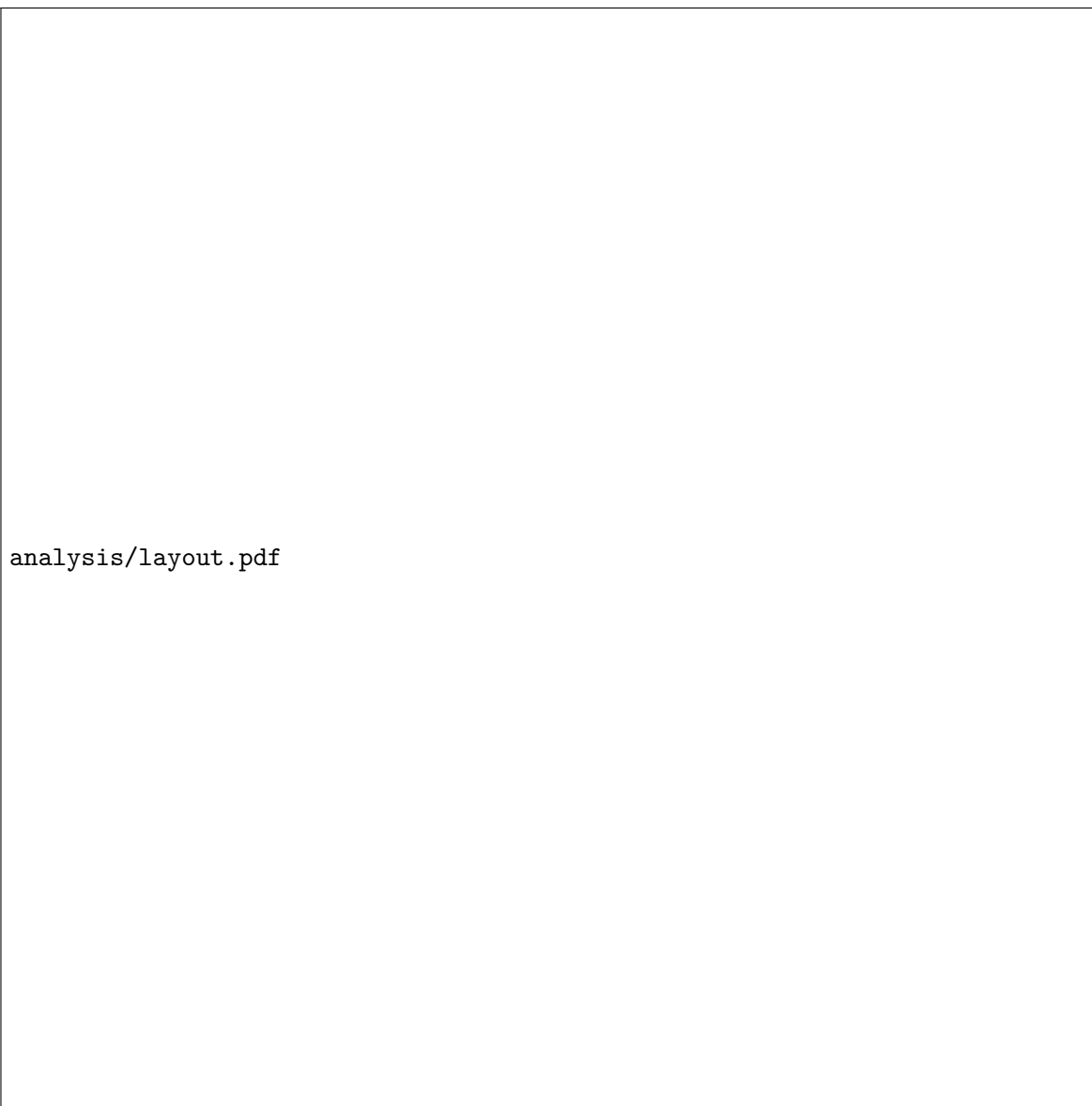
4 Startup reconstruction

4.1 Entry and pre-relocation hardware operation

The inspected entry performs the following actions:

1. At 0x0, write zero to VTTBR_EL2, followed by an instruction synchronization barrier.
2. Read HCR_EL2, OR in 0x400000000 and 0x80000000, write it back, and synchronize. These operations are observed; the report does not infer the incoming exception level or all platform-specific implications.
3. Form the current image address and load the preferred address.
4. Call 0x3cee0 from 0x28.
5. Compare current and preferred addresses and either enter relocation or continue at 0x174.

The helper at 0x3cee0 reads the implementation-specific register S3_3_C15_C7_0, tests its top bit, derives quantities from bitfields and MPIDR_EL1, and may update that register. One failure path



analysis/layout.pdf

Figure 1: Entropy and zero-byte fraction in 64 KiB windows. The dashed line marks the observed end of the main code-dense prefix, not a formal executable section boundary.

writes 0xa00a0005 to VA 0x3082b8030 and loops at 0x3cf50. Another path polls the top bit before returning at 0x3cf7c. The peripheral meaning and liveness of these operations are unresolved. It would be incorrect to label this helper as image signature verification or to replace it with an unconditional return.

4.2 Overlap-aware relocation

The relocation path obtains the linked image length, rounds it to 64 bytes, and compares source and destination intervals. It selects forward or backward copying for the overlap case. Rather than overwrite the code performing the copy, it copies a small copy routine to scratch VA 0x1fc000000, performs instruction-cache maintenance, and jumps to that scratch routine.

Offset	Reconstructed role
0x34–0x94	Length calculation, overlap decisions, scratch setup
0x9c–0xb8	Copy a routine to scratch, synchronize, branch there
0xd8–0x110	Forward copying in 64-byte iterations
0x110–0x148	Backward copying in 64-byte iterations
0x148–0x174	Zero source-only range, clear scratch, branch to destination

A behavior-level summary, *not* a literal C translation, is:

```
current = address_of_running_image;
preferred = literal_at_0x300;
span = round_up_64(literal_at_0x308 - preferred);
if (current != preferred) {
    select_overlap_safe_copy_direction();
    place_copy_routine_in_scratch(0x1fc000000);
    synchronize_instruction_stream();
    copy_image_in_64_byte_blocks(current, preferred, span);
    zero_old_source_bytes_not_overlapping_destination();
    zero_scratch_copy_routine();
    branch_to(preferred);
}
continue_at_offset_0x174();
```

The assembly clears non-overlapping portions of the old source region rather than blindly clearing the entire original interval after an overlapping move. Exact exception behavior and writable address ranges depend on the platform.

4.3 Zero initialization and runtime entry

At 0x174, the code masks exceptions with DAIFSet, writes an implementation-specific register, and sets VBAR_EL1 to candidate VA 0x1fc22b800, corresponding to file offset 0x1ab800. It then zeros three ranges in the following order:

Start VA	End VA, exclusive	Literal offsets
0x1fc044000	0x1fc048000	0x348, 0x350
0x1fc000000	0x1fc040000	0x328, 0x330
0x1fc407480	0x1fc483910	0x338, 0x340

The last range is consistent with zero-initialized runtime storage beyond the file-backed span. That interpretation is stronger than simply treating all addresses after end-of-file as invalid, but no original BSS section name or complete memory map is recovered.

The branch at 0x1d0 reaches 0x36a68. This routine clears TTBR0_EL1 and TTBR1_EL1, selects a stack, and sets `sp` to VA 0x1fc048000. It invokes several low-level helpers, updates control registers, and walks the pointer interval VA 0x1fc407470–0x1fc407478. The single pointer there resolves to file offset 0x45f60, called with BLRAAZ. A branch at 0x36b10 then enters 0x1b11e8.

Within that later routine, code at 0x1b14c4 references the string `main`; code at 0x1b14ec constructs and authenticates a callback for 0x7f484, passing it to 0x1bb150. The callback at 0x7f484 calls 0x3644, which reaches 0x3678. This supports a main-task setup interpretation, but does not by itself establish every scheduler transition.

```
0x000000 -> 0x03cee0 -> relocation or 0x000174
0x0001d0 -> 0x036a68
0x036b04 -> authenticated pointer -> 0x045f60
0x036b10 -> 0x1b11e8
0x1b14ec -> constructs callback 0x07f484 for "main"
0x07f490 -> 0x003644 -> 0x003678
```

The first, second, third, fourth, and last lines describe inspected calls or branches. The callback-construction line is a data-flow observation, not a claim that the scheduler immediately executes that callback.

5 Command records and dispatch

5.1 Recovered descriptor family

Sixteen records share a descriptor pointer to candidate VA 0x1fc2828a0 (file offset 0x2028a0). Each has two equal string pointers, an end pointer one byte beyond the name’s final character, and a code pointer at record relative offset 0x20. The observed 120-byte record footprint is:

Relative offset	Observed field; semantic labels are provisional
0x0, 0x8, 0x10	Name pointer, equal pointer, end pointer
0x18	Associated metadata pointer
0x20	Command handler pointer
0x28	Shared descriptor pointer
0x30–0x68	Runtime-related pointers, metadata, and zero fields
0x70	Small raw value, observed as 0 or 2

The pointer tuple resembles a bounded-pointer representation, but the original C types, compiler instrumentation, ownership rules, and exact field names have not been recovered. Values 0 and 2 are reported as raw flags; their meaning is *not* assumed to be privilege, availability, or argument count.

Command	Record offset	Handler offset	Raw flags
bootx	0x2030a8	0x1184	0x2
memboot	0x203120	0x1958	0x0
go	0x203198	0x2374	0x0
reboot	0x204258	0x4ea4	0x0
reset	0x2042d0	0x4ea4	0x0
bgcolor	0x204348	0x4eec	0x2
setpicture	0x2043c0	0x4f88	0x2
firmware	0x204438	0x4d9c	0x2
devicetree	0x2044b0	0x5368	0x2
getenv	0x204528	0x15defc	0x0
saveenv	0x2045a0	0x4ea0	0x0
setenv	0x204618	0x15dff4	0x0
setenvnp	0x204690	0x15e21c	0x0
clearenvp	0x204708	0x15e284	0x0
ramdisk	0x204780	0x53c8	0x2
rsepfirmware	0x2047f8	0x19fc94	0x0

This is the complete set found with this descriptor pattern, not a guarantee that it includes all console features, alternate parsers, or all commands available on every boot path. In particular, the `reboot` and `reset` records genuinely share the handler at `0x4ea4`.

5.2 Dispatch corroborates the handler field

The worker-like routine at `0x1d6664` accesses a current command object through a global context, checks a pointer against associated bounds, and loads a callback from that object at byte offset 32. It checks for null, loads arguments, and calls `BLRAAZ x8` at `0x1d66ac`. It stores the returned `w0` at context offset 12 and loops through additional synchronization helpers. This independently corroborates the record's `0x20` handler field.

```
repeat:
    call_state_helpers();
    descriptor = checked_pointer_from_context();
    callback = descriptor.field_at_byte_32;
    if (callback == 0) call_failure_helper_0x1de4d0();
    result = authenticated_call(callback,
        context.w0_arg, context.x1_arg, context.x2_arg,
        context.x3_arg, context.x4_arg);
    context.word_at_byte_12 = result;
    call_completion_helpers();
```

This is a schematic description. Helper names are analytical placeholders, and an ordinary `int command(int argc, char **argv)` prototype is not established by the observed register and bounds metadata.

5.3 Behavior supported by inspected handlers

bootx, 0x1184: Obtains a candidate memory region and calls 0x7fa3c with size 0x1e000000. A false result branches to the “Permission Denied” path at 0x13e0. Later code references tc-path, chosen, isp-horizon, and boot breadcrumbs. This is a boot-processing handler, not demonstrated arbitrary-address execution.

memboot, 0x1958: Reads filesize through helper 0x15d3a4. A zero value fails. Values at least 0x20000001 fail, so the local size gate permits 1 through 0x20000000, inclusive. The handler then checks the region using 0x7fa3c; subsequent code references device-tree, SEP, ramdisk, root-hash, and trust-cache paths.

go, 0x2374: Calls the same range predicate, then a parser-like routine at 0xc3df8. A nonzero parser result reaches “Memory image not valid”. The command name alone is not evidence that it executes arbitrary raw input without format or policy checks.

firmware, 0x4d9c: Contains argument checks, region/size acquisition, the shared range predicate, and a success tail branch to 0x32a0. Its failure paths report unavailable size or denied memory access. The complete accepted firmware formats are not reconstructed.

Environment commands: saveenv at 0x4ea0 is a direct branch to 0x15deb8. getenv, setenv, setenvvp, and clearenvp have separate handlers in the 0x15de00–0x15e300 area. Their exact persistence and authorization semantics require deeper tracing.

Other commands: bgcolor, setpicture, devicetree, ramdisk, and rsepfirmware have named records and code pointers. Their names support display, device-tree, ramdisk, and SEP-related roles; complete protocols and side effects are not claimed.

6 A locally reconstructed memory-range predicate

The complete short routine at 0x7fa3c–0x7fa84 is especially useful because several command handlers call it before processing a memory image. It consumes a starting address in x0 and length in x4; its other incoming argument registers are not used by this body.

It reads a flag byte at VA 0x1fc45ca52. If bit 0 is set, it returns 1. Otherwise it rejects unsigned addition overflow, a start below the lower bound at VA 0x1fc45ca60, or an end above the upper bound at VA 0x1fc45ca68. A source-level model of just this local computation is:

```
bool range_allowed(uint64_t start, uint64_t length,
                  uint8_t flag_byte,
                  uint64_t lower, uint64_t upper)
{
    if ((flag_byte & 1) != 0)
        return true;
    uint64_t end = start + length;
    if (end < start)
        return false;
    if (lower > start)
        return false;
    return end <= upper;
}
```

The parameters for the global values are introduced solely to make this model readable and independently checkable. They are not the original binary ABI. For the normal path, the predicate is:

$$\neg \text{carry}(\text{start} + \text{length}) \wedge \text{lower} \leq \text{start} \wedge \text{start} + \text{length} \leq \text{upper}.$$

This is a memory-range check, not signature verification. The presence of “Permission Denied” at its callers does not change that conclusion. The source of the bounds, circumstances setting the flag, and additional checks before or after this predicate are unresolved. No claim of a security bypass follows from observing its flag-controlled alternative.

For `memboot`, the first locally verified gates can therefore be summarized, with later processing deliberately left opaque:

```
size = read_filesize_or_zero();
if (size == 0)
    report("filesize variable invalid or not set, aborting");
else if (size >= 0x20000001)
    report("Combo image too large");
else if (!range_allowed_from_runtime_state(buffer, size))
    report("Permission Denied");
else
    continue_memboot_processing_not_fully_reconstructed();
```

The range predicate accepts zero length when its bounds permit; the separate `memboot` size test rejects zero before reaching it. Keeping these tests separate avoids inventing behavior in the common helper.

7 Automatic boot, recovery, and policy tables

7.1 Automatic boot has concrete input and dispatch gates

The routine beginning at `0x4874` reaches code that reads `upgrade-retry`, `auto-boot`, a selected command, and `bootdelay`. The relevant address constructions include:

Reference instruction	String offset	Name
<code>0x4944</code>	<code>0x29b649</code>	<code>upgrade-retry</code>
<code>0x4970</code>	<code>0x29b657</code>	<code>auto-boot</code>
<code>0x4a24</code>	<code>0x29b669</code>	<code>bootdelay</code>
<code>0x4b8c</code>	<code>0x29b673</code>	<code>boot-command</code>

At `0x4a7c`, a helper returns an input value. Comparisons at `0x4a80` and `0x4a84` test 13 and 10 (carriage return and line feed). Either value reaches the autoboot-abort message at `0x4aa8`. The timeout path multiplies the low 32 bits of the delay by 1,000,000, compares elapsed values, and calls another helper with 10,000 between polls. These constants suggest second-to-microsecond conversion. The follow-up audit in Section 10 traces the counter/frequency helpers; no hardware timebase measurement was performed.

After the wait, the code scans 56-byte records, compares names through helper `0x16ac10`, and calls a selected callback from record offset 32 using `BLRAAZ` at `0x4bc0`. A fallback indirect call appears

at 0x4c2c. Overflow, bounds, and null-pointer failure paths are explicit in this region. The exact precedence among upgrade recovery, one-time commands, normal boot, and all fallback cases is not fully reconstructed.

7.2 Boot-policy actions form a distinct descriptor family

The following named records use descriptor 0x2033f8, separate from the console-command descriptor. They must not be silently counted as additional console commands. Duplicate names have different callbacks in different record groups.

Embedded name	Record offset	Handler offset
upgrade	0x338fc8	0x6090
recover	0x339000	0x6da4
recover-once	0x339038	0x725c
fsboot	0x3390a8	0xbd0
upgrade	0x3390e0	0x5b04
recover	0x339118	0x6934
recover-once	0x339150	0x6d84
checkerboard	0x339188	0xbb0
repair	0x3391c0	0xf40
device-recovery	0x3391f8	0xf60

The named handlers imply filesystem boot, upgrade, recovery, diagnostics-like, repair, and device-recovery actions. Their interpretation as policy actions is corroborated by the 56-byte dispatch records, but a name is not a complete specification of the corresponding state machine.

7.3 Recovery starts named worker-like tasks

At 0x4488, the code references the message “Entering recovery mode, starting command prompt.” It next associates the name `command` with an authenticated callback to 0x1d6604, invoking a task-creation-like wrapper at 0x4ce0. A `poweroff` callback is prepared nearby. The `idle-off` setting controls another path that prepares an `idleoff` callback.

The callback at 0x1d6604 initializes state and loops through a service helper at 0x169e5c, updating a time-related value. A separate nearby routine, 0x1d6664, performs the authenticated command invocation described earlier. This is stronger evidence than the recovery message alone, but does not recover the transport protocol or prove unrestricted command access.

8 Other reconstructed subsystem entry points

8.1 Image-related callbacks

Fourteen records share descriptor 0x203300; the repeated groups retain their separate record addresses below. The callback associations are observed; the grouping as image-related processing is

inferred from their names and surrounding firmware references.

Embedded name	Record offset	Handler offset
iBootData	0x203480	0x163314
Trustcache	0x2034c8	0x1aa308
DCP	0x203510	0x140a14
RTBuddy	0x203558	0x18c81c
SPTM	0x2035a0	0x1a2b2c
Exclave	0x2035e8	0x1a353c
iBootData	0x203630	0x163314
Trustcache	0x203678	0x1aa308
DCP	0x2036c0	0x140a14
RTBuddy	0x203708	0x18c81c
WCHF	0x203750	0x14c1bc
RestoreANS	0x203798	0xb044
SPTM	0x2037e0	0x1a2b2c
Exclave	0x203828	0x1a353c

These give concrete starting points for analyzing iBootData, trust caches, display-related firmware, auxiliary processors, SPTM, and Exclave-related processing. No claim is made that the corresponding firmware payloads are fully present in this file or that every callback runs on the current platform.

8.2 Image4 and boot-policy evidence

Apple’s platform documentation describes iBoot as part of the verified boot chain. Its public security documentation also describes Image4 as an ASN.1-based secure-boot format [2, 3]. For *this* artifact, the following local evidence is more specific than that background:

Code reference	String offset	Observed text or topic
0x6278	0x29be22	verify boot manifest hash
0x280b4	0x29dedc	Missing Image4 validity callback diagnostic
0x296e0	0x29df39	Image4 property-capture callback registration diagnostic
0x29c84	0x29dfef	chosen/manifest-properties
0x29cf4	0x29e006	chosen/manifest-object-properties
0x29db0	0x29dfa6	Boot-manifest handoff without initialization diagnostic
0x2b270	0x29e3fb	boot-manifest-hash
0x1a75b0	0x2a4035	manifest-entitlements

Other strings name DeviceTree, BootKC, AuxKC, TrustCache, ExclaveOSIntegrityCatalog, and ExclaveOSTrustCache. Firmware-path strings mention SPTM, TXM, SEP, and other compo-

nents. These establish vocabulary and data-flow leads, *not* a verified end-to-end authentication implementation.

Certificate-related strings include Apple secure-boot root CA names at 0x2de113, 0x2dea93, and 0x2df222. Their presence does not establish certificate validity, actual trust-anchor selection, or successful verification of this binary. No cryptographic chain or signature was checked.

8.3 Storage, hardware, and runtime evidence

Storage-related references at 0x75760 onward name NVMe firmware, system configuration, NAND, and other devices. An `apfs` string appears at 0x381498, and filesystem boot has a named callback. These observations suggest storage and filesystem support, without reconstructing an on-disk format parser in full.

A further 38 named records share descriptor 0x205788 and point to USB/PHY/fabric tuning routines. Their names and targets are listed in Appendix A. They are hardware-tuning callback records, not shell commands. The literal “Apple USB Serial Interface” at 0x386950 and the command worker provide additional USB/console clues, but the USB request grammar is unresolved.

The entry and runtime code contain exception-vector setup, low-level control register operations, task-like callback construction, and authenticated indirect calls. PAC prologues and authenticated returns are consistent with return-address protection; Arm documents the relationship among PACIASP/PACIBSP and authenticated return instructions [5]. Their presence does not prove that all pointers, all control transfers, or all memory accesses are protected.

9 Deliverables and reproducibility

The report and its full evidence package are available in this workspace:

File	Purpose
<code>main.tex</code> , <code>main.pdf</code>	This manuscript and compiled report.
<code>analysis/reconstruct.py</code>	Hash-checked static extractor and offset-range decoder.
<code>analysis/linear_sweep.asm.gz</code>	All 924,958 aligned words, raw encodings, candidate addresses, and decoding where possible.
<code>analysis/selected_code.asm</code>	Bounded excerpts for the inspected startup, runtime, policy, predicate, and command regions.
<code>analysis/function_candidates.tsv</code>	Heuristic entries with their evidence classes.
<code>analysis/direct_calls.tsv</code>	Immediate BL edges and provisional caller indexing.
<code>analysis/indirect_branches.tsv</code>	Plain BR/BLR sites, excluding authenticated variants.

File	Purpose
<code>analysis/strings.tsv</code>	Printable runs with file offsets and candidate VAs.
<code>analysis/string_references.tsv</code>	Recoverable address-construction and absolute-pointer references.
<code>analysis/commands.tsv</code>	Sixteen command records and handler associations.
<code>analysis/named_callbacks.tsv</code>	Seventy-eight named records across four descriptor families.
<code>analysis/entropy.tsv</code>	Byte-distribution measurements.
<code>analysis/summary.json</code>	Hash, mapping, mechanical counts, and limitations.
<code>analysis/make_report_assets.py</code>	Recreates the included tables and layout plot.
<code>analysis/README.md</code>	Artifact guide and reproducibility instructions.

Run these commands from the workspace root:

```
python analysis/reconstruct.py
python analysis/make_report_assets.py
latexmk -pdf -interaction=nonstopmode -halt-on-error main.tex
```

A focused inspection without rebuilding the catalogs is:

```
python analysis/reconstruct.py --range 0x7fa3c 0x7fa84
```

The script intentionally rejects another binary hash because its base, code-region limit, and descriptor classifications are specific to this input. The evidence files use file offsets so that observations remain auditable without assuming a runtime load address.

10 Logic-bug audit

10.1 Result and limits of the claim

The follow-up audit identifies **two conditional arithmetic-risk candidates**, C01 and C02. Their truncating computations and numerical counterexamples are established from the instruction encodings. Neither has been demonstrated to violate a supported input contract on running hardware. Consequently, the count of *confirmed end-to-end logical defects or exploitable vulnerabilities in this audit is zero*, not two.

This distinction matters: a multiplication that wraps for an arbitrary integer is not automatically a bug if every permitted caller supplies a smaller value. Conversely, a downstream check cannot recover bits that an earlier narrow operation has discarded. The audit investigates both the arithmetic and the surrounding gates rather than treating every suspicious instruction as a flaw.

An exhaustive “every logical bug” claim is not supported. The raw binary lacks source contracts, complete types, the full platform model, and observed execution. Indirect control flow and external firmware remain unresolved. The following coverage records describe work actually performed, not a certification that the firmware is correct.

10.2 Audit coverage

`analysis/audit_binary.py` scans the complete previously identified code-dense prefix, file offsets 0x0–0x201b28, and records arithmetic, branch, and loop patterns. This pass uses LLVM features +v8.3a,+crypto. Enabling crypto decoding resolves, for example, the AESD words at 0x1f8154 and 0x1f81c4 that the original decoder configuration did not recognize. It does not establish which extensions are implemented by the actual hardware.

Mechanical coverage measure	Count
Aligned word slots scanned in the code-dense prefix	526,026
Slots decoded with the audit’s feature configuration	525,495
32-bit multiply-family patterns	1,869
Signed or unsigned division patterns	1,797
Variable-shift patterns	627
Indirect call patterns, including authenticated forms	3,973
Indirect jump patterns, including authenticated forms	338
Unconditional branches to their own address	14
Short backward branches containing reads, without calls	877
Named callback records considered	78
Distinct named callback entry addresses	71
Union of slots in callback walks without descending into calls	5,724
Slots visited by the interprocedural direct-flow walk	273,076
Unresolved indirect sites encountered by that direct-flow walk	502
Decode/range stops in that direct-flow walk	0

The counts of patterns are *not findings*. The prefix contains data as well as instructions. All con-

ditional edges are considered without proving feasibility. The callback walks follow direct branches and assume calls may return; this assumption can include adjacent code after an actually non-returning call. The larger direct-flow walk also follows direct callees, starting at the image entry and the 71 distinct named callbacks. It cannot follow an unresolved indirect jump and does not recover every indirect callee, exception edge, or asynchronous transition.

The 78 records comprise 16 command records, 14 image-callback records, 10 boot-policy action records, and 38 tunable records. These have respectively 15, 8, 10, and 38 distinct handler addresses. All records received the automated walk; this is *not* a claim that all handler bodies and transitive callees received complete manual semantic review.

The complete-file raw-word listing from the initial reconstruction remains available. The additional bug-pattern scan deliberately targets the identified code-dense prefix rather than pretending that decoding the opaque/data regions constitutes a semantic audit of those regions.

10.3 C01: autoboot delay narrows before comparison

Status: conditional candidate; arithmetic confirmed, permitted-input reachability unresolved.

The environment lookup at 0x4a40 calls 0x15d3a4, supplying a default value of 1. That getter returns a cached 64-bit value from an environment record at field offset 96, or the default when no record is found. No numerical clamp appears in that getter. At 0x4a48, the returned value is saved in x23. The crucial instruction sequence is:

```
0x004a70  mov  w8, #16960
0x004a74  movk  w8, #15, lsl #16
0x004a78  mul   w23, w23, w8
...
0x004a90  sub   x8, x0, x19
0x004a94  cmp   x8, x23
0x004a98  b.hi  0x004ae8
```

The constructed constant is 1,000,000. The W-register form retains only 32 result bits, unlike a widened multiplication [4]. The comparison therefore uses

$$\text{threshold} = (\text{delay} \cdot 1,000,000) \bmod 2^{32}.$$

Delay input	Exact product	Retained product
4,294	4,294,000,000	4,294,000,000
4,295	4,295,000,000	32,704
5,000	5,000,000,000	705,032,704

Thus the first overflowing positive integral delay is 4,295. If such a value is permitted to reach this path, increasing the requested delay can make the wait much shorter. The time getter at 0x1c9a4c reaches counter-reading and frequency-conversion helpers at 0x500, 0x460, and 0x48c; the latter uses CNTFRQ_ELO and a divisor of 1,000,000. This supports nominal microsecond units, but the actual counter frequency was not measured.

Reachability qualification. The inspected console setter calls a name filter at 0x3a28 through 0x15e0d4. Its four-name table at 0x2038c0 contains `auto-boot`, `boot-args`, `debug-uart`s, and `pwr-path`; it does not contain `bootdelay`. A straightforward console assignment is therefore not an established path to the example. Other environment initialization/import paths and the permitted domain of this setting are not fully traced.

Remediation if the wider input domain is supported. Use a checked 64-bit product, or explicitly reject values exceeding the supported limit before multiplication. Merely widening the subsequent comparison cannot restore the discarded bits. No firmware patch has been attempted.

10.4 C02: register-value conversion truncates before a width check

Status: conditional candidate; arithmetic and downstream check ordering confirmed, operand provenance and supported range unresolved.

The routine at 0x186d84 saves incoming `x4` in `x22` and `x3` in `x23`. A wrapper at 0x18608c obtains these inputs from two 32-bit fields at structure offsets 8 and 12, after checking the structure pointer against associated bounds. A pointer to that wrapper appears at file offset 0x384250. This provides a concrete data source and callback reference, but does not identify an external transport or establish that untrusted input can populate the fields.

The conversion and subsequent calls include:

```
0x186de4  mov  w8, #13824
0x186de8  movk  w8, #366, lsl #16
0x186dec  mul   w22, w22, w8
0x186df0  mul   w23, w23, w8
...
0x186e24  mov   x2, x23
0x186e2c  bl    0x17ee1c
...
0x186e38  mov   x2, x22
0x186e40  bl    0x17ee1c
```

The multiplier is 24,000,000. The largest input whose exact product fits an unsigned 32-bit value is 178:

$$178 \cdot 24,000,000 = 4,272,000,000,$$

whereas

$$179 \cdot 24,000,000 = 4,296,000,000 \longrightarrow 1,032,704 \pmod{2^{32}}.$$

One channel-specific branch can replace `x23` with the constant 10; that does not impose a general bound on the independent `x22` input. The exact hardware meaning and units of these values remain unconfirmed.

The register-access helper at 0x17ee1c subsequently checks whether its value argument has a nonzero high word:

```
0x17ee44  lsr   x8, x2, #32
0x17ee48  cbz   x8, 0x17ee50
0x17ee4c  cbz   w23, 0x17efa8
```

For these callers, `w3` is zero. Nevertheless, a wrapped product from `MUL W` always arrives with a zero high word, so this later validation cannot detect the earlier overflow. This is a concrete example of validation occurring after information has been lost, not an assertion that the helper itself has a faulty high-word check.

Reachability qualification. A defect requires the original input contract to allow an out-of-range value, or an upstream validator to fail to exclude it. Hardware register limits, trusted configuration generation, and all producers of the structure have not been recovered. A real-world trigger, security consequence, and severity are therefore not established.

Remediation if needed. Check the input against the destination’s representable range or preserve a full-width product until the existing width check. Widening the hardware write itself is not necessarily valid; the register may genuinely be 32 bits wide.

10.5 Suspicious patterns not promoted to findings

Range overflow at 0x7fa3c: The routine uses `ADDS` and an unsigned carry branch before comparing against both bounds. Boundary models did not produce a counterexample to the reconstructed normal-path interval predicate. Zero-length acceptance is a separate policy matter; `memboot` rejects zero size before this call.

Apparent gaps in accumulated ranges: The updater at 0x7f968 computes the minimum start and maximum end, so its representation would cover a gap if fed disjoint intervals. However, the observed immediate call at 0x7f820 registers one region after resetting state, and the call at 0x2e564 requests a reset rather than adding a second region. No immediate branch or literal pointer to another updater caller was found. A hypothetical two-range example is not evidence of a reachable bug. Indirect aliases and other writers remain a coverage limitation.

The range-check override flag: The flag is part of shared runtime state, not a demonstrated user-controlled parameter. A limited address-reference scan identifies 55 accesses in its surrounding state window. This does not establish every writer or the policy that authorizes the override; it is not reported as an authentication bypass.

Variable shifts in tunable dispatch: In the shared dispatcher at 0x739ac, 0x73a00 rejects selector values above 8 before one variable shift. At 0x73ac8–0x73adc, subtracting one, checking the unsigned result against 7, and testing mask 139 restrict another selector to widths {1,2,4,8}. A shift instruction alone is not an unchecked index. The indirect callbacks and actual register semantics still require platform-specific analysis.

Relocation erasure overrun: A fabricated unaligned source/destination difference could make a 64-byte zeroing loop look dangerous. Here the source is constructed with `ADRP` plus zero, and the preferred destination is page-aligned. Their difference is consequently a multiple of 4,096; the linked span is also a multiple of 64. Under those observed address-construction assumptions, the source-only erased interval is 64-byte aligned and disjoint from the new image. The unaligned counterexample does not apply to this entry path.

Intentional multiplication wrap at 0x1d73e4: The constant 0xaaaaaab is the multiplicative inverse of 3 modulo 2^{32} . In this sequence, it recovers an index previously multiplied by 3, with an earlier bound of 0x10000. Wrap is part of the computation, not a discarded large allocation size. The corresponding identity was checked for all 65,536 indices in that range.

Small-domain signed arithmetic at 0x94394: The multiplicand comes from the difference of two extracted 8-bit values. Its mathematical range is $[-255, 255]$, so multiplying by 5,000,000 remains inside the signed 32-bit interval. A large multiplier without input-domain analysis would have produced a false alarm here.

Console and image-processing gates: The inspected dispatcher validates a descriptor pointer and checks its callback for null before the authenticated call at 0x1d66ac. `memboot`’s nonzero/maximum-size checks and `setpicture`’s 0x400000 byte-size limit are also present. These observations rule out specific missing-check claims, not every error in the called parsers or the full secure-boot chain.

10.6 Unresolved liveness and semantic risks

The loop at 0x3cf70–0x3cf78 repeatedly reads an implementation-specific register and waits for a bit. Other compact polling sequences include 0x4afc8–0x4afcc and 0x4b01c–0x4b020. There is no in-loop software timeout in those particular instruction sequences. If the awaited condition never changes, they do not progress normally. Whether this is an unacceptable hang depends on hardware guarantees, interrupt behavior, external watchdogs, and intended fail-stop policy, none of which is fully specified. The audit therefore records this as *unresolved liveness behavior*, not hundreds of confirmed infinite-loop bugs.

The 14 self-branches likewise include halt-like paths and exception-vector regions. A branch to itself can be deliberate after shutdown, a fatal error, or an unexpected exception. No blanket removal or timeout patch is justified from their presence alone.

Not exhaustively established are the Image4 authentication predicates, every manifest error path, USB/storage request grammars, all hardware-tuning side effects, concurrent state updates, and final OS/auxiliary-firmware handoff. The 502 unresolved indirect sites encountered in the direct-flow walk provide concrete remaining work rather than an implicit assumption of safety.

10.7 Reproducible checks and artifacts

`analysis/audit_models.py` checks the input hash and eight relevant instruction words, then evaluates independent local mathematical properties:

Model check	Cases
Range predicate, including 64-bit edges and override states	46,244
Tunable width selector	258
Intentional modular-inverse multiplication	65,536
Page-aligned relocation erasure intervals	2,048
Illustrative delay/register-value conversions	10

These are *model checks*, not execution of the firmware and not proof that all callers satisfy the modeled preconditions. Passing them cannot validate unrelated code or eliminate the documented reachability gaps.

Audit artifact	Contents
<code>analysis/audit/coverage.json</code>	Exact scan configuration, counts, and limitations.
<code>analysis/audit/evidence.asm</code>	Short annotated excerpts supporting the candidates and reviewed checks.
<code>analysis/audit/instruction_patterns.tsv</code>	Whole-prefix syntactic arithmetic and control-flow leads, not findings.
<code>analysis/audit/constant_multiply_leads.tsv</code>	Local large-constant multiplication leads and arithmetic thresholds.
<code>analysis/audit/callback_coverage.tsv</code>	One coverage row per named callback record.
<code>analysis/audit/callback_calls.tsv</code>	Direct calls encountered in each callback walk.
<code>analysis/audit/direct_flow_intervals.tsv</code>	Slots visited by the larger direct-flow walk, as half-open intervals.
<code>analysis/audit/direct_flow_unresolved.tsv</code>	Remaining indirect sites encountered by that walk.
<code>analysis/audit/short_read_loops.tsv</code>	Short syntactic read-loop candidates requiring contextual review.
<code>analysis/audit/range_state_references.tsv</code>	Limited address-based references to the shared range state.
<code>analysis/audit/model_checks.json</code>	Check counts and the exact arithmetic examples.

Reproduce from the workspace root:

```
python -B analysis/reconstruct.py
python -B analysis/audit_binary.py
python -B analysis/audit_models.py
python -B analysis/make_report_assets.py
latexmk -pdf -interaction=nonstopmode -halt-on-error main.tex
```

The firmware is not modified. No payload, hardware probe, or exploit is needed for these local arithmetic and structural checks.

11 What remains before a complete reconstruction

A complete, behaviorally faithful replacement cannot be claimed from this pass. The outstanding work is specific:

1. **Recover actual code boundaries and control flow.** Separate literal pools and jump tables from instructions, recursively trace reachable blocks, and resolve authenticated and table-driven indirect calls. Candidate-entry and BL counts are not substitutes for this work.
2. **Recover data types and calling conventions.** The repeated bounds and metadata pointers require typed analysis. Optimized shared fragments and nonstandard argument tuples make naive C decompilation misleading.

3. **Finish boot-policy and image verification paths.** Identify all accepted image formats, manifest checks, callback contracts, error propagation, and handoff conditions. The short memory predicate is only one local check.
4. **Identify opaque data and missing dependencies.** Explain the high-entropy region and the eight-byte file/span discrepancy. Obtain relevant external firmware, platform descriptions, and the original image container where available.
5. **Validate against execution.** An appropriate emulator or authorized hardware trace is needed to validate register assumptions, memory-mapped I/O, scheduler behavior, command availability, timing, and the final OS handoff.
6. **Build and compare a replacement, if required.** That would require new implementation work, tests, and platform validation. It would still not recover the original comments, identifiers, build system, or exact source text.

Result. The complete file has a reproducible mechanical inventory, and selected central behaviors have been reconstructed with precise locations. The relocation path, memory-range predicate, named handlers, and selected autoboot/dispatch transitions have stronger evidence than string-only subsystem attributions. Unanalyzed driver bodies, opaque data, and unresolved security and runtime state are not presented as recovered source.

A Hardware-tuning callback inventory

These 38 named records use descriptor 0x205788. Their record offsets are preserved in `analysis/named_callbacks.tsv`; the table shows handler entry offsets only. Names are embedded labels, not invented source symbols.

Embedded name	Handler offset
tunable_DRD_USB31_DBG_DEFAULT	0x11c450
tunable_DRD_USB31_GBL_DEFAULT	0x11c4fc
tunable_DRD_USB31_GBL_HOST	0x11c530
tunable_LINK_REGS_DEFAULT	0x11c7a0
tunable_DRD_USB31_CFG_HOST	0x11c948
tunable_BULK_FABRIC_DEFAULT	0x11dbb8
tunable_PIPE_HANDLER_DEFAULT	0x11dddc
tunable_AUSBC_DEBUG_DEFAULT	0x11dd54
tunable_ATCOAXI2AF	0xe4750
tunable_ATCOAXI2AF_LOCK_LOCK	0xe47ec
tunable_ATCOAXI2AF_WR_DIS	0xe4884
tunable_ATC_FABRIC	0x11c6f4
tunable_ATC_COMMON_CFG	0x11c350
tunable_USB2PHY_HOST	0x11dc40
tunable_USB2PHY_DEV	0x11dc74
tunable_USB2PHY_REG_DFLT	0x124a7c
tunable_AUX_TOP	0xe4018
tunable_AUSCMN_SHM	0x11e380
tunable_AUSCMN_DIG	0x11e06c
tunable_ACIOPHY_PLL_TOP	0xe4394
tunable_AUSPLL_CORE	0x11e780
tunable_CLKMON_CFG	0x122f60
tunable_CIO3PLL_CORE	0x122cf0
tunable_CIO3PLL_TOP	0x122e54
tunable_ACIOPHY_LANE_USBC0	0xe4234
tunable_ACIOPHY_LANE_USBC1	0xe42b4
tunable_CIO_SHIM	0xe4110
tunable_ACIOPHY_TOP	0xe4494
tunable_LN0_RX_TOP_USB_DFLT	0x1211a4
tunable_LN0_RX_EQ_USB_EQA	0x11f338
tunable_LN0_TX_TOP_USB_DFLT	0x1224fc
tunable_LN0_TX_SHM_USB_DFLT	0x121c10
tunable_LN0_RX_SHM_USB_DFLT	0x11fbb0
tunable_LN1_RX_TOP_USB_DFLT	0x121268
tunable_LN1_RX_EQ_USB_EQA	0x11f3b8
tunable_LN1_TX_TOP_USB_DFLT	0x12257c
tunable_LN1_TX_SHM_USB_DFLT	0x121c90
tunable_LN1_RX_SHM_USB_DFLT	0x11fbe4

References

- [1] LLVM Project. *LLVM C API: Disassembler*. Documentation for `LLVMCreateDisasmCPUFeatures` and `LLVMDisasmInstruction`. Consulted September 13, 2026; the local analysis uses the installed LLVM 19 shared library.
- [2] Apple. *Apple Platform Security: Boot process for iPad and iPhone devices*. Consulted September 13, 2026. General boot-chain background, not identification or authentication of the supplied artifact.
- [3] Apple Security Research. *Private Cloud Compute Security Guide: Hardware Root of Trust*. Consulted September 13, 2026. Referenced only for the general description of Image4, not to classify this artifact as a Private Cloud Compute image.
- [4] Arm. *ARMv8-A A64 ISA Overview*. Discussion of W/X register views and instruction operand widths. Consulted September 13, 2026. Local arithmetic claims are additionally anchored to the supplied instruction words and reproducible numerical models.
- [5] Arm. *Code reuse attacks: the compiler story*. Discussion of pointer-authentication prologues and authenticated returns. Consulted September 13, 2026.